



Projet PC3R - Find Me An Event

Melisa Butterworth & Bogdan Styn

Mai 2026

Table des matières

1	Introduction	4
2	Présentation générale de l'application	4
2.1	Concept	4
2.2	Fonctionnement général	4
3	Architecture du système	5
3.1	Architecture globale	5
3.2	Structure du projet	5
3.2.1	Backend Go	5
3.2.2	Frontend React	6
4	Technologies utilisées	6
4.1	Pourquoi Go ?	6
4.2	Pourquoi React ?	7
5	Backend et API REST	7
5.1	Serveur HTTP Go	7
5.2	API REST	7
5.3	Communication JSON	7
6	Authentification et gestion des sessions	8
6.1	Authentification	8
6.2	Sessions	8
6.3	Sécurité CORS	8
7	Intégration des API externes	9
7.1	API OpenData Paris	9
7.2	API ip-api.com	9
8	Base de données PostgreSQL	10
8.1	Tables principales	10
8.2	Contraintes SQL	10
9	Gestion concurrente et communication client-serveur	10
9.1	Gestion concurrente des requêtes HTTP	10
9.2	Requêtes asynchrones côté client	10
9.3	Synchronisation des données	11
9.4	Scalabilité	11
10	Frontend React	11
10.1	Application monopage	11
10.2	Routes protégées	12
10.3	Gestion globale de l'état	12
10.4	Interface utilisateur	12
11	Tests	12
12	Cas d'utilisation	12
12.1	Création et connexion d'un compte	12
12.2	Découverte d'un événement	13
12.3	Consultation des avis et interactions sociales	13

1 Introduction

Dans le cadre du cours PC3R, nous avons développé une application web nommée *Find Me An Event* permettant de découvrir des événements à proximité à Paris. L’objectif principal du projet était de concevoir une application web moderne reposant sur une architecture client/serveur, utilisant des communications asynchrones et intégrant des API externes dynamiques.

L’utilisateur peut créer un compte, se connecter, être géolocalisé automatiquement grâce à son adresse IP, puis consulter des événements situés dans un rayon et une catégorie définis. Les événements sont affichés un par un dans une interface inspirée des applications de rencontres modernes. L’utilisateur peut voter “like” ou “unlike”, consulter les avis d’autres utilisateurs et publier ses propres commentaires.

Le projet respecte les contraintes imposées par le sujet de micro-projet PC3R :

- serveur développé en Go avec le package standard `net/http` ;
- communications HTTP asynchrones en JSON ;
- utilisation d’une base de données PostgreSQL ;
- intégration d’API externes dynamiques ;
- contenu généré par les utilisateurs ;
- architecture web complète client/serveur.

Le lien vers le dépôt GitHub où se trouve notre code est le suivant : dépôt github

2 Présentation générale de l’application

2.1 Concept

Find Me An Event est une plateforme de découverte d’événements culturels à Paris. L’application récupère des événements publics depuis l’API OpenData de la ville de Paris et les propose à l’utilisateur selon sa position géographique et la catégorie choisie.

L’utilisateur peut :

- s’inscrire et se connecter ;
- être géolocalisé automatiquement ;
- choisir un rayon de recherche ;
- filtrer par différentes catégories comme cinéma, concert, exposition, théâtre, art, sport ou famille ;
- consulter des événements proches ;
- liker ou disliker des événements ;
- consulter les statistiques de votes ;
- écrire des commentaires et des avis ;
- accéder à son historique personnel.

2.2 Fonctionnement général

Le client React communique avec le serveur Go via une API REST JSON. Le serveur interroge :

- PostgreSQL pour les données utilisateurs ;
- l’API OpenData Paris pour les événements ;
- l’API `ip-api.com` pour la géolocalisation.

Toutes les communications sont réalisées via HTTP et la majorité des échanges sont asynchrones.

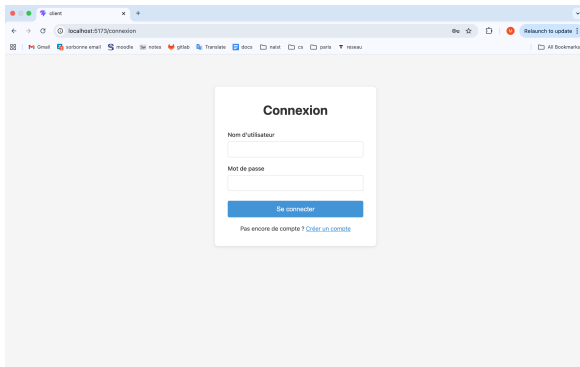


FIGURE 1 – Page de connexion

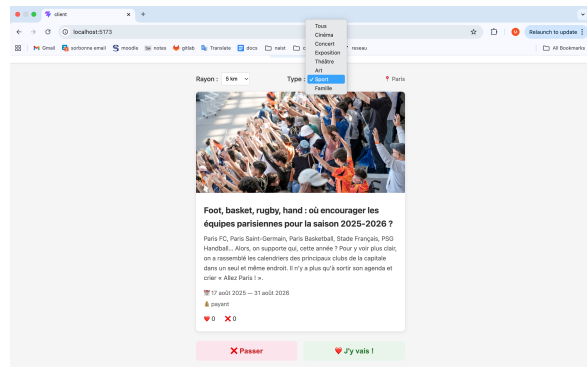


FIGURE 2 – Un événement trouvé avec différents filtres

3 Architecture du système

3.1 Architecture globale

Le système suit une architecture client/serveur classique en trois couches :

- Frontend React
- Backend Go
- Base de données PostgreSQL

Le frontend est responsable de l'interface utilisateur et des interactions. Le backend gère la logique métier, l'authentification, les appels aux API externes et la communication avec PostgreSQL.

3.2 Structure du projet

3.2.1 Backend Go

Le serveur est organisé en plusieurs fichiers spécialisés :

- `main.go` : point d'entrée du serveur, définition des routes REST et démarrage du serveur HTTP ;
- `config.go` : gestion de la configuration et des variables d'environnement ;
- `db.go` : connexion PostgreSQL et création automatique des tables ;
- `auth.go` : gestion de l'inscription, connexion et déconnexion des utilisateurs ;
- `utilisateur.go` : modèle SQL et opérations sur les utilisateurs ;
- `session.go` : création et gestion des sessions utilisateurs ;
- `middleware.go` : protection des routes nécessitant une authentification ;
- `geolocalisation.go` : communication avec l'API `ip-api.com` ;
- `evenements.go` : récupération des événements depuis l'API OpenData ;
- `vote.go` et `avis.go` : modèles SQL des votes et commentaires utilisateurs ;
- `handler_api.go` : gestion des routes liées à la géolocalisation et aux événements ;
- `handler_votes.go` : gestion des votes, avis et historique utilisateur ;
- `cors.go` : configuration du middleware CORS entre React et Go ;
- `server_test.go` : tests unitaires des fonctions pures du serveur (sans dépendance à la base).

3.2.2 Frontend React

Le client React utilise une architecture par composants :

- `main.jsx` : point d'entrée de l'application React ;
- `App.jsx` : gestion des routes et des pages protégées ;
- `api.js` : centralisation des appels HTTP asynchrones ;
- `AuthContext.jsx` : fournisseur global de l'état d'authentification utilisateur ;
- `useAuth.js` : hook de consommation du contexte d'authentification ;
- `app.css` : styles globaux et interface responsive.

Le dossier `components/` contient les composants réutilisables :

- `Navigation.jsx` : barre de navigation principale ;
- `CarteEvenement.jsx` : affichage des événements et des votes ;
- `FormulaireAvis.jsx` : publication des avis et commentaires.

Le dossier `pages/` contient les principales pages de l'application :

- `Connexion.jsx` : page de connexion ;
- `Inscription.jsx` : création de compte ;
- `Decouverte.jsx` : découverte et vote des événements ;
- `DetailEvenement.jsx` : affichage détaillé des événements ;
- `Historique.jsx` : historique des votes utilisateurs ;
- `NotFound.jsx` : page 404 pour les URLs inconnues.

`vite.config.js` : configuration du proxy entre React et le serveur Go.

4 Technologies utilisées

Composant	Technologie
Backend	Go + <code>net/http</code>
Frontend	React + Vite
Base de données	PostgreSQL
API externe événements	OpenData Paris
API géolocalisation	<code>ip-api.com</code>
Communication	HTTP + JSON

4.1 Pourquoi Go ?

Go a été choisi pour plusieurs raisons :

- gestion native de la concurrence avec les goroutines ;
- serveur HTTP performant dans la bibliothèque standard ;
- simplicité du langage ;
- bonne gestion des applications réseau.

L'utilisation de `net/http` permet également de respecter les contraintes du projet sans utiliser de framework web.

4.2 Pourquoi React ?

React facilite la création d'une interface dynamique et réactive. Les composants permettent de séparer clairement les différentes parties de l'interface.

Le système de routing permet également de créer une application monopage (SPA).

5 Backend et API REST

5.1 Serveur HTTP Go

Le serveur repose entièrement sur le package standard `net/http`.

Les routes sont définies dans `main.go` à l'aide d'un `ServeMux`. Le routeur de Go 1.22+ permet de filtrer directement par méthode HTTP et de récupérer les paramètres de chemin sans dépendance externe.

Exemple :

```
mux.HandleFunc("POST /api/login", handlerConnexion(db))
```

Chaque route correspond à une fonctionnalité précise.

5.2 API REST

L'API suit une logique REST simple : chaque ressource (utilisateurs, événements, votes, avis) est exposée derrière un ensemble cohérent de routes.

Méthode	Route	Description
POST	/api/register	Inscription d'un nouvel utilisateur
POST	/api/login	Connexion (création de session)
POST	/api/logout	Déconnexion (destruction de session)
GET	/api/me	Profil de l'utilisateur connecté
GET	/api/location	Géolocalisation par IP
GET	/api/events	Liste des événements (lat, lon, radius, categorie)
GET	/api/events/next	Prochain événement non encore voté
POST	/api/events/{id}/vote	Enregistre un like ou un unlike
DELETE	/api/events/{id}/vote	Retire le vote
GET	/api/events/{id}/stats	Compteurs de likes / unlikes
GET	/api/events/{id}/reviews	Liste des avis publiés
POST	/api/events/{id}/reviews	Publication d'un avis
GET	/api/me/history	Historique des votes de l'utilisateur

5.3 Communication JSON

Toutes les données sont échangées au format JSON.

Exemple de vote :

```
{
  "vote": "like",
  "titre": "Concert Jazz Paris"
}
```

6 Authentification et gestion des sessions

6.1 Authentification

L'application permet aux utilisateurs de créer un compte avec un nom, une adresse mail et un mot de passe. Lors de l'inscription, le nom d'utilisateur ainsi que l'adresse mail doivent être uniques afin d'éviter les doublons dans la base de données. L'adresse mail doit également respecter un format valide pour limiter les erreurs de saisie. Les mots de passe ne sont pas stockés en clair dans la base de données : ils sont sécurisés grâce à l'algorithme de hashage `bcrypt` avant d'être enregistrés. Le projet utilise les fonctions `GenerateFromPassword` et `CompareHashAndPassword` du package `golang.org/x/crypto/bcrypt`, avec un facteur de coût de 10, afin de garantir un stockage sécurisé des mots de passe utilisateurs.

6.2 Sessions

Après la connexion d'un utilisateur, le serveur génère un token de session aléatoire permettant d'identifier de manière unique la session active. Cette logique est principalement implémentée dans le fichier `session.go`, qui utilise les bibliothèques standard de Go pour générer des tokens sécurisés et gérer les cookies HTTP. Le système permet de sécuriser l'accès aux routes protégées sans stocker directement les informations sensibles de l'utilisateur côté client.

Le token de session est :

- généré aléatoirement (32 octets via `crypto/rand`) lors de la connexion dans `auth.go` ;
- associé à l'utilisateur connecté ;
- stocké dans PostgreSQL via les requêtes définies dans `session.go` ;
- lié à une date d'expiration afin de limiter la durée de validité de la session ;
- envoyé au navigateur sous forme de cookie `HttpOnly` avec `SameSite=Lax` grâce au package standard `net/http`, ce qui empêche son vol par un script malveillant et bloque la majorité des attaques CSRF.

Le middleware d'authentification, défini dans `middleware.go`, intervient ensuite avant chaque accès à une route protégée :

- il récupère le token présent dans le cookie HTTP ;
- vérifie son existence dans la base PostgreSQL ;
- contrôle que la session n'a pas expiré ;
- injecte l'identifiant de l'utilisateur dans le contexte de la requête pour les handlers en aval ;
- autorise ou refuse l'accès selon la validité de la session.

Ce mécanisme permet de sécuriser les fonctionnalités nécessitant une authentification, comme les votes, les commentaires ou l'historique utilisateur.

6.3 Sécurité CORS

Comme le client React (port 5173) et le serveur Go (port 8080) tournent sur des origines différentes en développement, le navigateur exige une politique CORS explicite. Le middleware `cors.go` n'autorise qu'une liste blanche d'origines (`http://localhost:5173` et `http://127.0.0.1:5173`) plutôt que de refléter aveuglément l'origine appelante. Combiné à l'envoi des cookies de session, cela évite qu'un site tiers puisse déclencher des requêtes authentifiées au nom d'un utilisateur connecté.

7 Intégration des API externes

7.1 API OpenData Paris

L'application utilise l'API OpenData « Que faire à Paris » afin de récupérer dynamiquement des événements culturels. Les données sont récupérées par le fichier `evenements.go`, qui construit une requête ODSQL envoyée à l'API.

Les événements sont filtrés selon :

- la position géographique de l'utilisateur, récupérée via l'API `ip-api.com`;
- le rayon de recherche choisi par l'utilisateur (en kilomètres) ;
- la catégorie choisie (recherchée dans le titre, la description et le chapeau via l'opérateur `search` d'ODSQL) ;
- les dates et informations disponibles dans l'API OpenData.

Paramètre	Utilisation
Position géographique	Permet de rechercher les événements proches de l'utilisateur.
Rayon de recherche	Définit la distance maximale des événements affichés.
Catégorie	Restreint le flux aux événements correspondant au mot-clé choisi.
Dates des événements	Permet de récupérer uniquement les événements disponibles.

TABLE 3 – Paramètres utilisés pour filtrer les événements

Le serveur construit dynamiquement une requête ODSQL à partir de ces paramètres afin de récupérer uniquement les événements situés à proximité de l'utilisateur. Les apostrophes contenues dans la catégorie sont échappées avant l'insertion dans la requête, pour empêcher toute injection dans le filtre ODSQL.

7.2 API ip-api.com

Cette API permet de géolocaliser automatiquement l'utilisateur à partir de son adresse IP. Le fichier `geolocalisation.go` interroge l'API `ip-api.com` afin de récupérer plusieurs informations, notamment la latitude, la longitude, la ville et le pays de l'utilisateur.

Information récupérée	Utilisation
Latitude	Calcul de la position géographique de l'utilisateur.
Longitude	Recherche des événements à proximité.
Ville	Affichage d'informations géographiques adaptées.
Pays	Localisation générale de l'utilisateur.

TABLE 4 – Informations récupérées via l'API ip-api.com

Ces données sont ensuite utilisées par le serveur pour rechercher les événements situés à proximité via l'API OpenData Paris. En développement, lorsque l'IP du client est privée (127.0.0.1, 192.168.x.x) et que `ip-api.com` ne peut pas la résoudre, le serveur retombe automatiquement sur les coordonnées de Paris pour que l'application reste utilisable.

8 Base de données PostgreSQL

8.1 Tables principales

- **utilisateurs** : informations des utilisateurs ;
- **sessions** : sessions actives ;
- **votes** : likes et unlikes ;
- **avis** : commentaires et notes.

8.2 Contraintes SQL

Le projet utilise plusieurs contraintes SQL :

- clés primaires ;
- clés étrangères ;
- contraintes **UNIQUE** ;
- contraintes **CHECK**.

Exemple :

```
CHECK (vote IN ('like', 'unlike'))
```

Le projet utilise également :

```
UNIQUE(utilisateur_id, evenement_id)
```

afin d'empêcher plusieurs votes pour un même événement.

9 Gestion concurrente et communication client-serveur

9.1 Gestion concurrente des requêtes HTTP

L'un des principaux intérêts du langage Go est sa gestion native de la concurrence grâce aux goroutines. Le serveur HTTP fourni par le package standard **net/http** exécute automatiquement chaque requête entrante dans une goroutine distincte, ce qui permet au serveur de traiter plusieurs clients simultanément sans bloquer l'application.

Ainsi, plusieurs utilisateurs peuvent en même temps :

- se connecter ;
- consulter des événements ;
- voter ;
- publier des commentaires ;
- récupérer leur historique personnel.

Cette approche est particulièrement adaptée à une application web interactive, car certaines opérations peuvent prendre du temps, notamment les appels aux API externes (**ip-api.com** et **OpenData Paris**) ou les requêtes PostgreSQL. Grâce aux goroutines, une requête lente n'empêche pas le traitement des autres utilisateurs connectés.

9.2 Requêtes asynchrones côté client

Le frontend React communique avec le serveur via des appels **fetch** asynchrones centralisés dans **api.js**. Les requêtes sont envoyées sans recharger complètement la page, ce qui améliore fortement la fluidité de l'interface et l'expérience utilisateur.

Par exemple :

- les événements sont chargés dynamiquement depuis le serveur ;
- les votes like/unlike sont envoyés instantanément ;
- les avis utilisateurs apparaissent sans rafraîchissement manuel ;
- l'historique peut être récupéré indépendamment des autres pages.

Cette architecture correspond également aux contraintes du sujet, qui demandent l'utilisation de communications AJAX et de réponses JSON entre le client et le serveur.

9.3 Synchronisation des données

Les votes et les commentaires étant accessibles à plusieurs utilisateurs simultanément, il est nécessaire de garantir la cohérence des données stockées dans PostgreSQL. Le projet utilise pour cela plusieurs contraintes SQL et mécanismes de validation côté serveur.

Par exemple, la table des votes contient la contrainte :

```
UNIQUE(utilisateur_id, evenement_id)
```

Cette contrainte empêche un utilisateur de voter plusieurs fois pour le même événement, même si plusieurs requêtes concurrentes sont envoyées presque simultanément. PostgreSQL joue donc un rôle important dans la synchronisation et la protection des données partagées entre utilisateurs. Lorsqu'un utilisateur change d'avis, le serveur utilise un `INSERT ... ON CONFLICT DO UPDATE` (upsert PostgreSQL) pour mettre à jour atomiquement le vote existant sans race condition.

9.4 Scalabilité

L'architecture choisie permet de supporter efficacement plusieurs utilisateurs simultanés. Le modèle concurrent de Go permet de traiter un grand nombre de requêtes HTTP en parallèle avec une faible consommation mémoire grâce aux goroutines. Les connexions PostgreSQL sont mutualisées par le driver SQL utilisé par Go, ce qui évite d'ouvrir une nouvelle connexion à chaque requête.

Enfin, l'API REST reste principalement stateless : chaque requête contient les informations nécessaires à son traitement via les cookies de session et les paramètres HTTP. Cette organisation facilite l'extension future du projet et permettrait plus facilement un déploiement sur plusieurs serveurs si nécessaire.

10 Frontend React

10.1 Application monopage

Le frontend fonctionne comme une SPA (*Single Page Application*).

React Router est utilisé pour gérer :

- connexion ;
- inscription ;
- découverte ;
- détail événement ;
- historique ;
- page 404 pour toute URL inconnue.

10.2 Routes protégées

Certaines pages nécessitent une authentification.

Le composant `RouteProtegee` vérifie qu'un utilisateur est connecté avant d'afficher la page demandée ; à l'inverse, `RoutePublique` redirige vers la page d'accueil un utilisateur déjà connecté qui tenterait d'accéder à la page de connexion ou d'inscription.

10.3 Gestion globale de l'état

L'état d'authentification est partagé via un `Context` React. Le fournisseur (`AuthProvider` dans `AuthContext.jsx`) tente une reprise automatique de session au montage en appelant `/api/me` ; le cookie étant `HttpOnly`, seul un aller-retour serveur permet de savoir si l'utilisateur est encore connecté. Le hook `useAuth` (extrait dans `useAuth.js`) est ensuite consommé par les composants pour accéder à l'utilisateur courant et aux fonctions `connecter`, `inscrire` et `deconnecter`.

10.4 Interface utilisateur

L'interface a été conçue pour être simple et responsive.

Le système de découverte d'événements s'inspire des interfaces de swipe modernes.

11 Tests

Le projet inclut une suite de tests Go unitaires (`server_test.go`) couvrant les fonctions pures du serveur, sans aucune dépendance à PostgreSQL ni aux API externes :

- génération du token de session (longueur, unicité) ;
- extraction de l'IP cliente, y compris le cas d'un `X-Forwarded-For` chaîné ;
- comportement du middleware CORS (préflight `OPTIONS` et requête réelle) ;
- helpers de réponse JSON et d'erreur ;
- conversion d'un enregistrement OpenData vers la structure `Evenement` (cas nominal, JSON invalide, absence de coordonnées) ;
- chargement de la configuration depuis l'environnement.

Les tests se lancent avec `go test ./...` depuis le dossier `server/`.

12 Cas d'utilisation

12.1 Création et connexion d'un compte

Lors de sa première utilisation, l'utilisateur crée un compte en renseignant un nom d'utilisateur, une adresse mail valide et un mot de passe. Le serveur vérifie que le nom et l'adresse mail sont uniques avant de stocker le mot de passe sous forme hashée avec `bcrypt`. Après connexion, une session sécurisée est créée grâce à un token stocké dans PostgreSQL et envoyé au navigateur via un cookie HTTP.

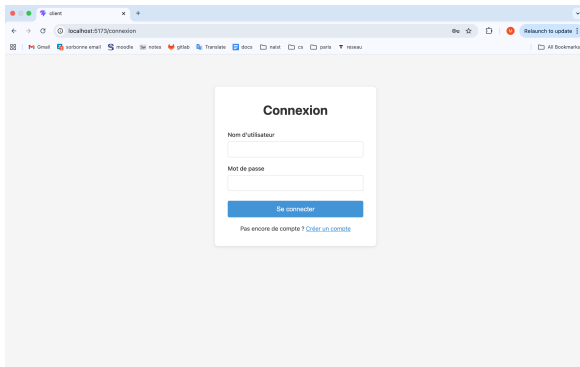


FIGURE 3 – Page de connexion

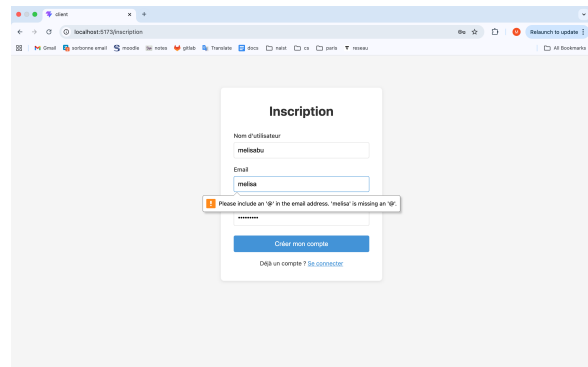


FIGURE 4 – Page d'inscription et avertissement en cas d'adresse e-mail invalide

12.2 Découverte d'un événement

Un utilisateur se connecte à l'application avec son nom et mot de passe. Après une géolocalisation automatique via son adresse IP, il choisit un rayon de recherche, par exemple 5 kilomètres, ainsi qu'une catégorie d'événements comme cinéma, concert, théâtre ou exposition. Le serveur interroge alors l'API OpenData Paris et récupère les événements correspondant aux critères sélectionnés.

Les événements sont ensuite affichés un par un dans une interface inspirée des applications de rencontre modernes. L'utilisateur peut rapidement parcourir les propositions et choisir de liker ou de passer un événement jusqu'à trouver une activité qui l'intéresse.

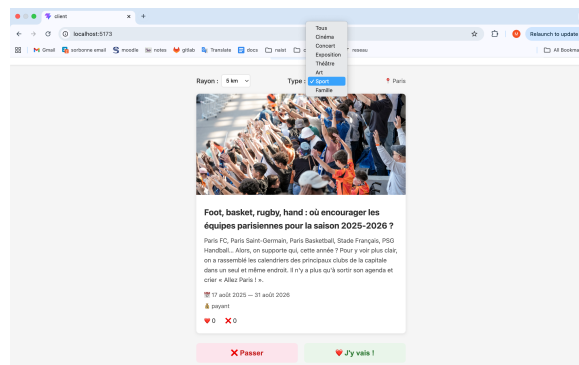


FIGURE 5 – Page montrant un événement et des catégories

12.3 Consultation des avis et interactions sociales

Lorsqu'un utilisateur "like" un événement, il peut accéder à une page détaillée contenant des informations complètes comme l'adresse, les dates, le prix, la description ou encore le nombre total de likes et d'unlikes. Les utilisateurs peuvent également consulter les avis publiés par les autres participants, attribuer une note sous forme d'étoiles et laisser leurs propres commentaires afin de partager leur expérience.

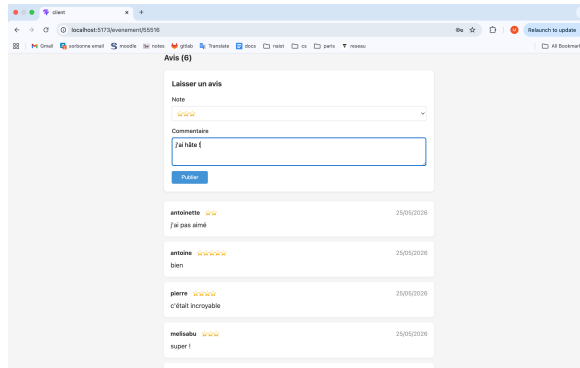


FIGURE 6 – Page montrant les avis des autres utilisateurs

13 Conclusion

Le projet *Find Me An Event* nous a permis de mettre en pratique les principaux concepts étudiés dans le cadre du cours PC3R.

Nous avons développé une application web complète reposant sur :

- une architecture moderne client/serveur ;
- des communications asynchrones ;
- une base de données relationnelle ;
- plusieurs API externes ;
- les capacités concurrentes du langage Go.

Enfin, cette expérience nous a permis d'acquérir une meilleure compréhension du développement full-stack et des problématiques liées aux applications web modernes.